Parte 1 - Escalabilidad vertical

1. Diríjase a el Portal de Azure y a continuación cree una maquina virtual con las características básicas descritas en la imágen 1 y que corresponden a las siguientes:
    - Resource Group = SCALABILITY_LAB
    - Virtual machine name = VERTICAL-SCALABILITY
    - Image = Ubuntu Server
    - Size = Standard B1ls
    - Username = scalability_lab
    - SSH publi key = Su llave ssh publica

→ Creamos un grupo de recursos

Datos básicos

Suscripción	Azure for Students
Grupo de recursos	SCALABILITY_LAB
Nombre de máquina virtual	VERTICAL-SCALABILITY
Región	East US 2
Opciones de disponibilidad	Zona de disponibilidad
Opciones de zona	Zona autoseleccionada
Zona de disponibilidad	1
Tipo de seguridad	Estándar
Imagen	Ubuntu Server 24.04 LTS - Gen2
Arquitectura de VM	x64
Tamaño	Standard B1ls (1 vcpu, 0.5 GiB de memoria)
Habilitar hibernación	No
Tipo de autenticación	Clave pública SSH
Nombre de usuario	scalability_lab
Formato de clave SSH	RSA
Nombre de par de claves	VERTICAL-SCALABILITY_key
Puertos de entrada públicos	SSH, HTTP
Azure de acceso puntual	No

Discos

< Anterior | Siguiente > | Crear

Generar un par de claves nuevo

Un par de claves SSH contiene una clave pública y una privada. **Azure no almacena la clave privada.** Una vez creado el recurso de clave SSH, no podrá volver a descargar la clave privada. Más información

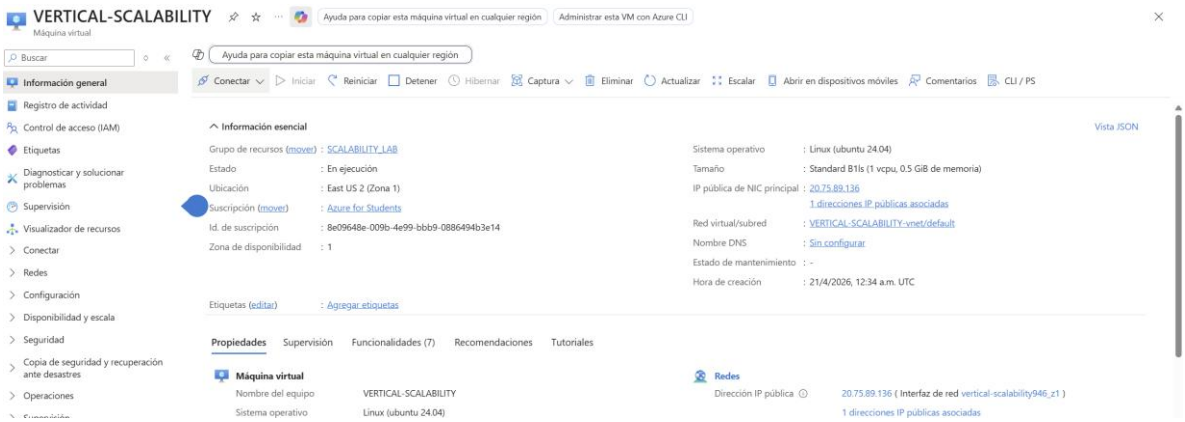
Descargar la clave privada y crear el recurso

Volver a la creación de una máquina virtual

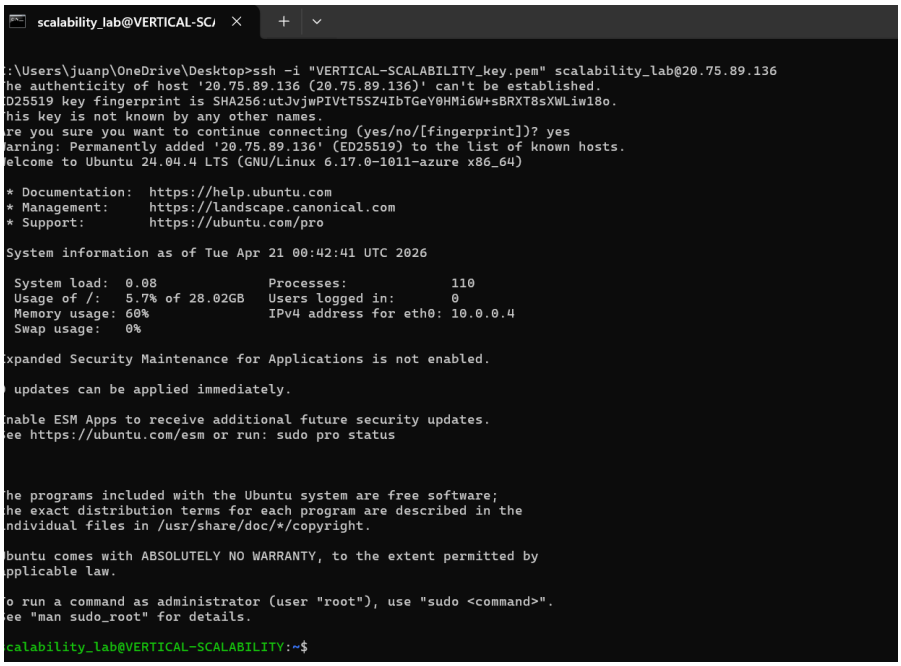
2. Para conectarse a la VM use el siguiente comando, donde las x las debe remplazar por la IP de su propia VM (Revise la sección "Connect" de la virtual machine creada para tener una guía más detallada).

ssh scalability_lab@xxx.xxx.xxx.xxx

→ Buscamos la ip de nuestra maquina en el dashboard en el Azure, es 20.75.89.136



- Luego, establecemos la conexión con la máquina



3. Instale node, para ello siga la sección *Installing Node.js and npm using NVM* que encontrará en este enlace.

→ Procedemos a instalar NVM, luego activar NVM en nuestra sesión actual y por ultimo instalar LTS de Node.js y verificamos la instalación.

```
scalability_lab@VERTICAL-SCALABILITY:~$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 16555  100 16555    0     0   269k      0 --:--:-- --:--:-- --:--:--  274k
=> Downloading nvm from git to '/home/scalability_lab/.nvm'
=> Cloning into '/home/scalability_lab/.nvm'...
remote: Enumerating objects: 424, done.
remote: Counting objects: 100% (424/424), done.
remote: Compressing objects: 100% (351/351), done.
remote: Total 424 (delta 59), reused 188 (delta 45), pack-reused 0 (from 0)
Receiving objects: 100% (424/424), 414.13 KiB | 10.90 MiB/s, done.
Resolving deltas: 100% (59/59), done.
* (HEAD detached at FETCH_HEAD)
  master
=> Compressing and cleaning up git repository

=> Appending nvm source string to /home/scalability_lab/.bashrc
=> Appending bash_completion source string to /home/scalability_lab/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"  # This loads nvm bash_completion
scalability_lab@VERTICAL-SCALABILITY:~$ source ~/.bashrc
scalability_lab@VERTICAL-SCALABILITY:~$ nvm --version
0.39.7
scalability_lab@VERTICAL-SCALABILITY:~$ nvm install --lts
Installing latest LTS version.
Downloading and installing node v24.15.0...
Downloading https://nodejs.org/dist/v24.15.0/node-v24.15.0-linux-x64.tar.xz...
######################################################################################################### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v24.15.0 (npm v11.12.1)
Creating default alias: default -> lts/* (-> v24.15.0)
scalability_lab@VERTICAL-SCALABILITY:~$ node -v
v24.15.0
scalability_lab@VERTICAL-SCALABILITY:~$ npm -v
11.12.1
```

4. Para instalar la aplicación adjunta al Laboratorio, suba la carpeta FibonacciApp a un repositorio al cual tenga acceso y ejecute estos comandos dentro de la VM:
   - git clone <your_repo>
   - cd <your_repo>/FibonacciApp
   - npm install

```
scalability_lab@VERTICAL-SCALABILITY:~$ git clone https://github.com/JuanCaballero9778/LAB09_ARSW
Cloning into 'LAB09_ARSW'...
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 37 (delta 3), reused 34 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (37/37), 3.55 MiB | 31.32 MiB/s, done.
Resolving deltas: 100% (3/3), done.
scalability_lab@VERTICAL-SCALABILITY:~$ cd LAB09_ARSW
scalability_lab@VERTICAL-SCALABILITY:~/LAB09_ARSW$ cd FibonacciApp
scalability_lab@VERTICAL-SCALABILITY:~/LAB09_ARSW/FibonacciApp$ npm install
npm warn old lockfile
npm warn old lockfile The package-lock.json file was created with an old version of npm,
npm warn old lockfile so supplemental metadata must be fetched from the registry.
npm warn old lockfile
npm warn old lockfile This is a one-time fix-up, please be patient...
npm warn old lockfile

added 51 packages, and audited 52 packages in 5s

7 vulnerabilities (3 low, 4 high)

To address all issues, run:
  npm audit fix
```

5. Para ejecutar la aplicación puede usar el comando npm FibinacciApp.js, sin embargo una vez pierda la conexión ssh la aplicación dejará de funcionar. Para evitar ese compartamiento usaremos *forever*. Ejecute los siguientes comando dentro de la VM.
   - node FibonacciApp.js

→ Para garantizar la disponibilidad continua de la aplicación, es necesario evitar la ejecución en primer plano con node FibonacciApp.js, ya que esta depende de una sesión SSH activa y se cerrará al desconectarse. En su lugar, aquí instalamos forever, una herramienta que instala el proceso como un *daemon* (proceso de segundo plano). Al ejecutar forever start FibonacciApp.js, delegamos la gestión al sistema operativo, lo que permite que la aplicación persista independientemente de la conexión del usuario.

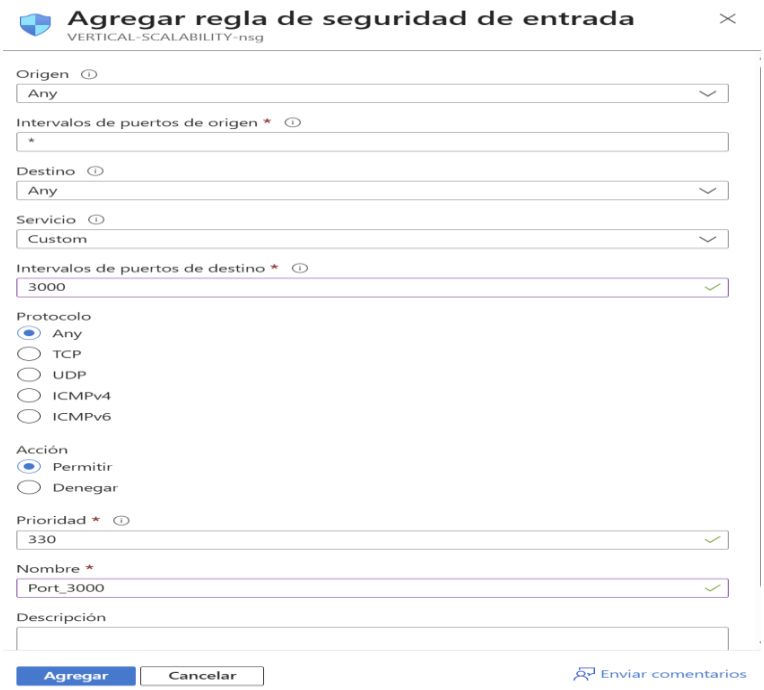
```
scalability_lab@VERTICAL-SCALABILITY:~/LAB09_ARSW/FibonacciApp$ npm install -g forever
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru
-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehen
sive and powerful.
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated urix@0.1.0: Please see https://github.com/lydell/urix#deprecated
npm warn deprecated source-map-url@0.4.1: See https://github.com/lydell/source-map-url#deprecated
npm warn deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#deprecated
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security v
ulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be pur
chased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated source-map-resolve@0.5.3: See https://github.com/lydell/source-map-resolve#deprecated

added 289 packages in 20s

50 packages are looking for funding
  run `npm fund` for details
scalability_lab@VERTICAL-SCALABILITY:~/LAB09_ARSW/FibonacciApp$ forever start FibonacciApp.js
warn:    --minUptime not set. Defaulting to: 1000ms
warn:    --spinSleepTime not set. Your script will exit if it does not stay up for at least 1000ms
info:    Forever processing file: FibonacciApp.js
(node:3900) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:3900) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(node:3900) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() inste
ad.
```

6. Antes de verificar si el endpoint funciona, en Azure vaya a la sección de *Networking* y cree una *Inbound port rule* tal como se muestra en la imágen. Para verificar que la aplicación funciona, use un browser y user el endpoint http://xxx.xxx.xxx.xxx:3000/fibonacci/6. La respuesta debe ser The answer is 8.

→ Ahora agregamos regla de seguridad de entrada, para añadir y poder abrir el puerto 3000



- Ahora verificamos que podamos acceder desde el browser y se pueda ver que la aplicación este funcionando

7. La función que calcula en enésimo número de la secuencia de Fibonacci está muy mal construido y consume bastante CPU para obtener la respuesta. Usando la consola del Browser documente los tiempos de respuesta para dicho endpoint usando los siguintes valores:

    1. 1000000
    2. 1010000
    3. 1020000
    4. 1030000
    5. 1040000
    6. 1050000
    7. 1060000
    8. 1070000
    9. 1080000
    10. 1090000

→

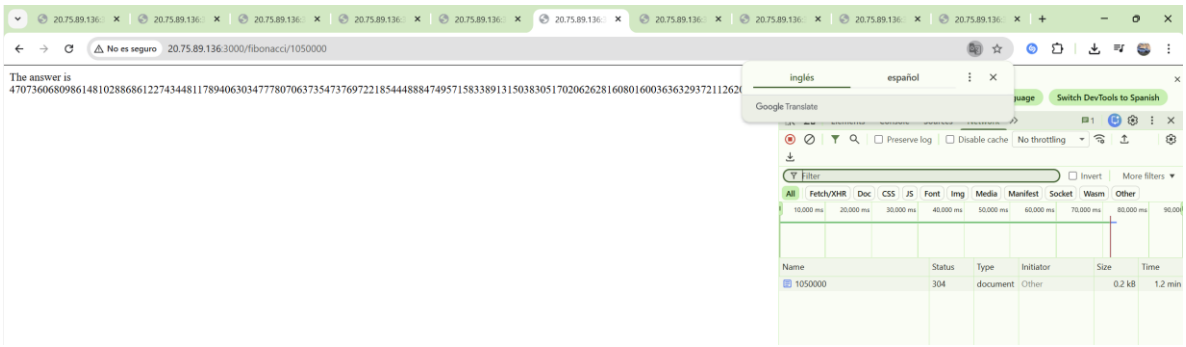
No es seguro
20.75.89.136:3000/fibonacci/1000000
The answer is
DevTools is now available in Spanish
Name
Status
Type
Initiator
Size
Time
1000000
304
document
Other
0.2 kB
13.04 s

No es seguro
20.75.89.136:3000/fibonacci/1010000
The answer is
DevTools is now available in Spanish
Name
Status
Type
Initiator
Size
Time
1010000
(pending)
document
Other
0.0 kB
Pending

No es seguro
20.75.89.136:3000/fibonacci/1020000
The answer is
DevTools is now available in Spanish
Name
Status
Type
Initiator
Size
Time
1020000
304
document
Other
0.2 kB
34.95 s

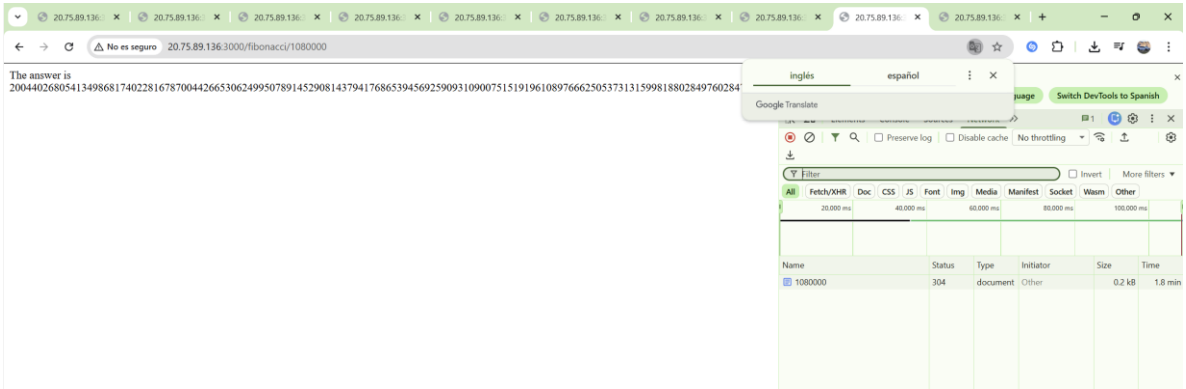
No es seguro
20.75.89.136:3000/fibonacci/1030000
The answer is
inglés
español
Google Translate
Name
Status
Type
Initiator
Size
Time
1030000
304
document
Other
0.2 kB
47.04 s

No es seguro
20.75.89.136:3000/fibonacci/1040000
The answer is
625712708408207380957509511912655096469979914959956769350294973003039605938438684487875835526496906393766778499998566764671943730973410005 9
DevTools is now available in Spanish
Don't show again
Always match Chrome's language
Switch DevTools to Spanish
Elements
Console
Sources
Network
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
Status
Type
Initiator
Size
Time
1040000
304
document
Other
0.2 kB
59.02 s


No es seguro
20.75.89.136:3000/fibonacci/1050000
The answer is
inglés
español
Google Translate
Switch DevTools to Spanish
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
Status
Type
Initiator
Size
Time
1050000
304
document
Other
0.2 kB
1.2 min

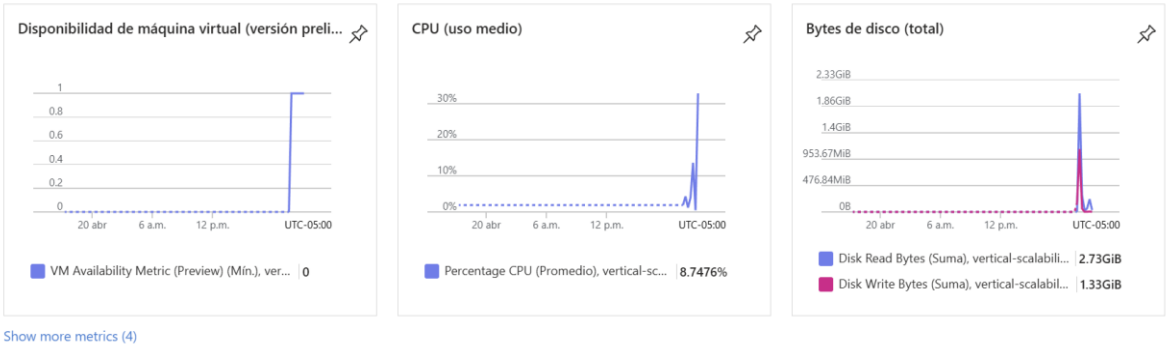
No es seguro
20.75.89.136:3000/fibonacci/1060000
The answer is
inglés
español
Google Translate
Switch DevTools to Spanish
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
Status
Type
Initiator
Size
Time
1060000
304
document
Other
0.2 kB
1.7 min

No es seguro
20.75.89.136:3000/fibonacci/1070000
The answer is
inglés
español
Google Translate
Switch DevTools to Spanish
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
Status
Type
Initiator
Size
Time
1070000
304
document
Other
0.2 kB
2.1 min



8. Dírijase ahora a Azure y verifique el consumo de CPU para la VM. (Los resultados pueden tardar 5 minutos en aparecer).

→

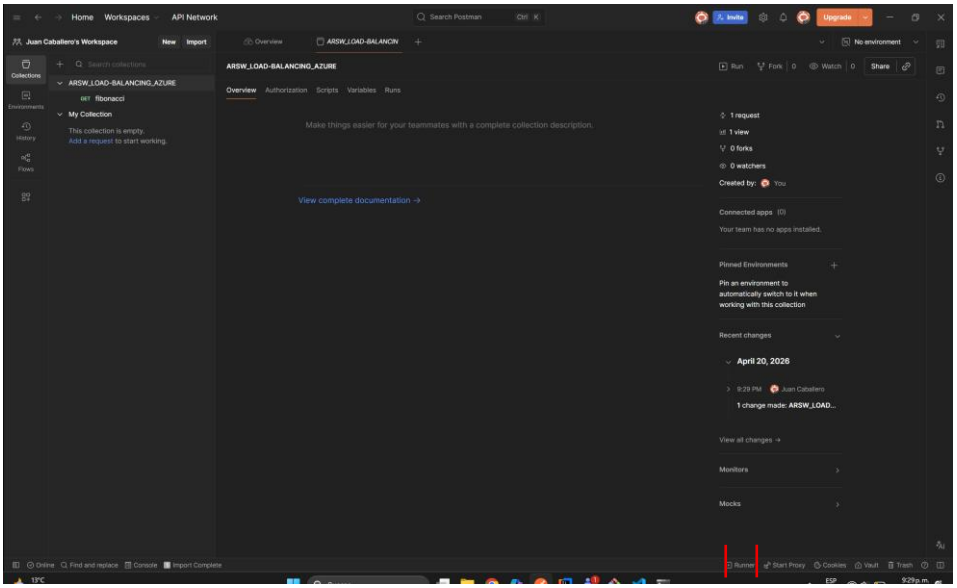


Ahora usaremos Postman para simular una carga concurrente a nuestro sistema. Siga estos pasos.

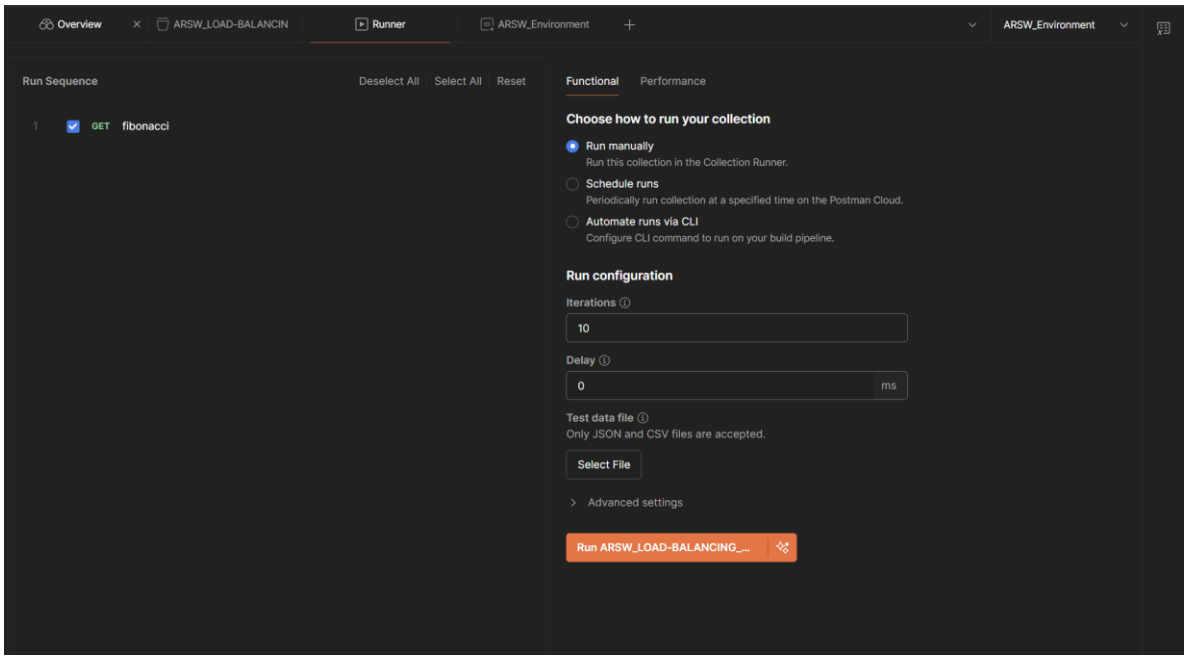
- Instale newman con el comando npm install newman -g. Para conocer más de Newman consulte el siguiente enlace.
- Diríjase hasta la ruta FibonacciApp/postman en una maquina diferente a la VM.

- Para el archivo [ARSW_LOAD-BALANCING_AZURE].postman_environment.json cambie el valor del parámetro VM1 para que coincida con la IP de su VM.
- Ejecute el siguiente comando.

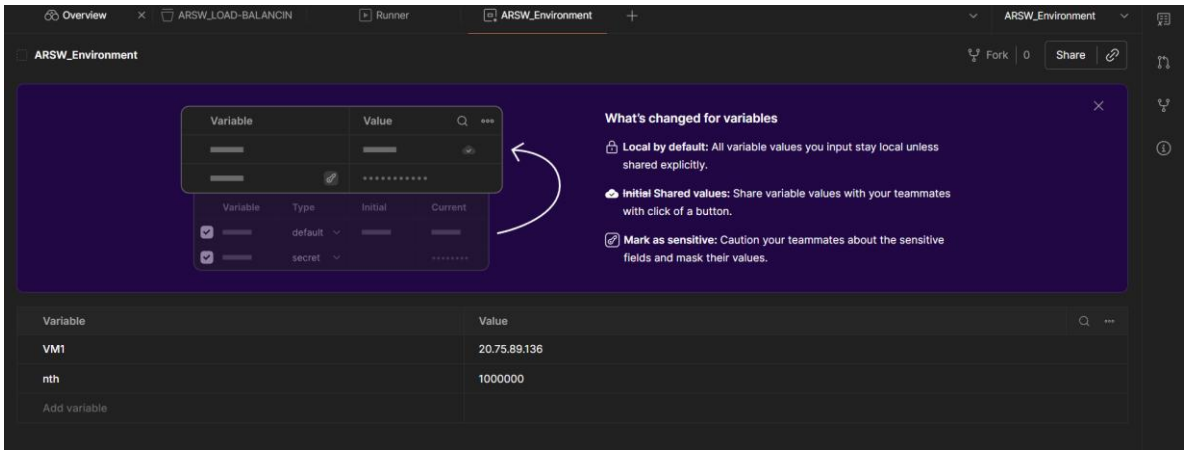
→ Utilice la app de escritorio de postman para tener una visualización mejor de la información, para ello importamos dentro de la aplicación el archivo .json ARSW_LOAD-BALANCING_AZURE.postman_collection



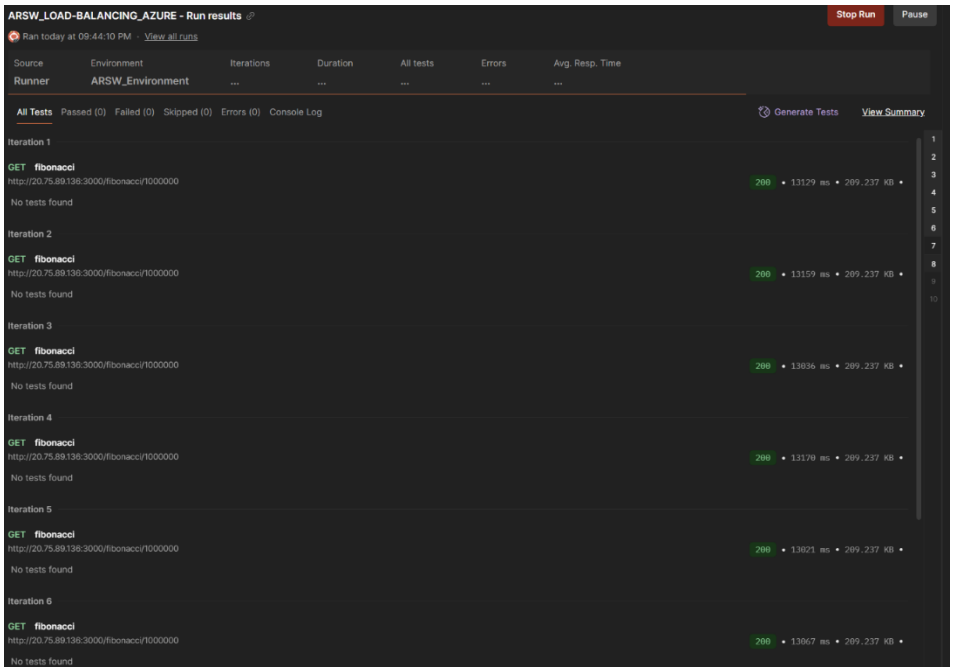
- Ahpra le damos click en runner.



- Agregamos los secretos para la ip y el valor de prueba en enviroment.



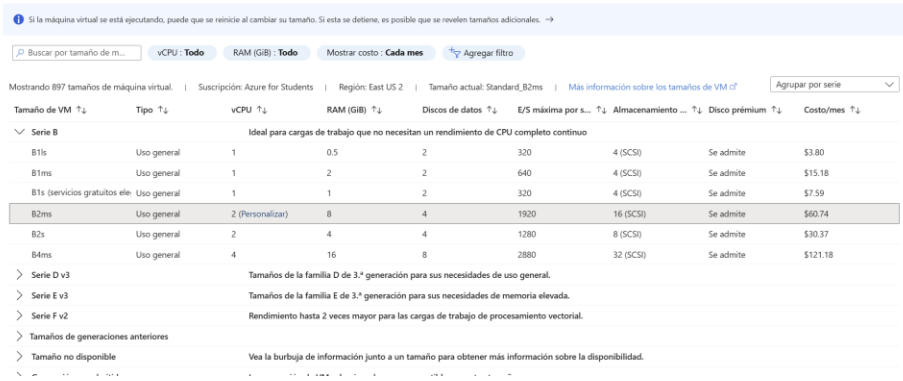
- Ahora procedemos a darle run (el botón naranja), para simular la carga concurrente

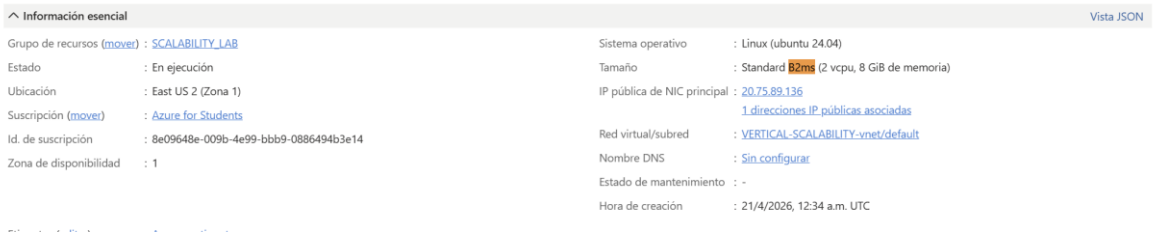


- La prueba realizada demuestra que la aplicación actual presenta un rendimiento limitado debido a la naturaleza monohilo del *Event Loop* de Node.js. Ante una carga de trabajo intensiva, cada petición bloquea el hilo principal durante aproximadamente 13 segundos. Esta arquitectura de 'bloqueo' implica que el sistema no puede atender múltiples peticiones en paralelo de manera eficiente, lo que resulta en un tiempo de espera acumulativo proporcional al número de usuarios concurrentes. Por lo tanto, se concluye que el escalamiento vertical es insuficiente para esta demanda, siendo necesario implementar una arquitectura de balanceo de carga para distribuir el cómputo entre múltiples instancias.

9. La cantidad de CPU consumida es bastante grande y un conjunto considerable de peticiones concurrentes pueden hacer fallar nuestro servicio. Para solucionarlo usaremos una estrategia de Escalamiento Vertical. En Azure diríjase a la sección *size* y a continuación seleccione el tamaño B2ms.

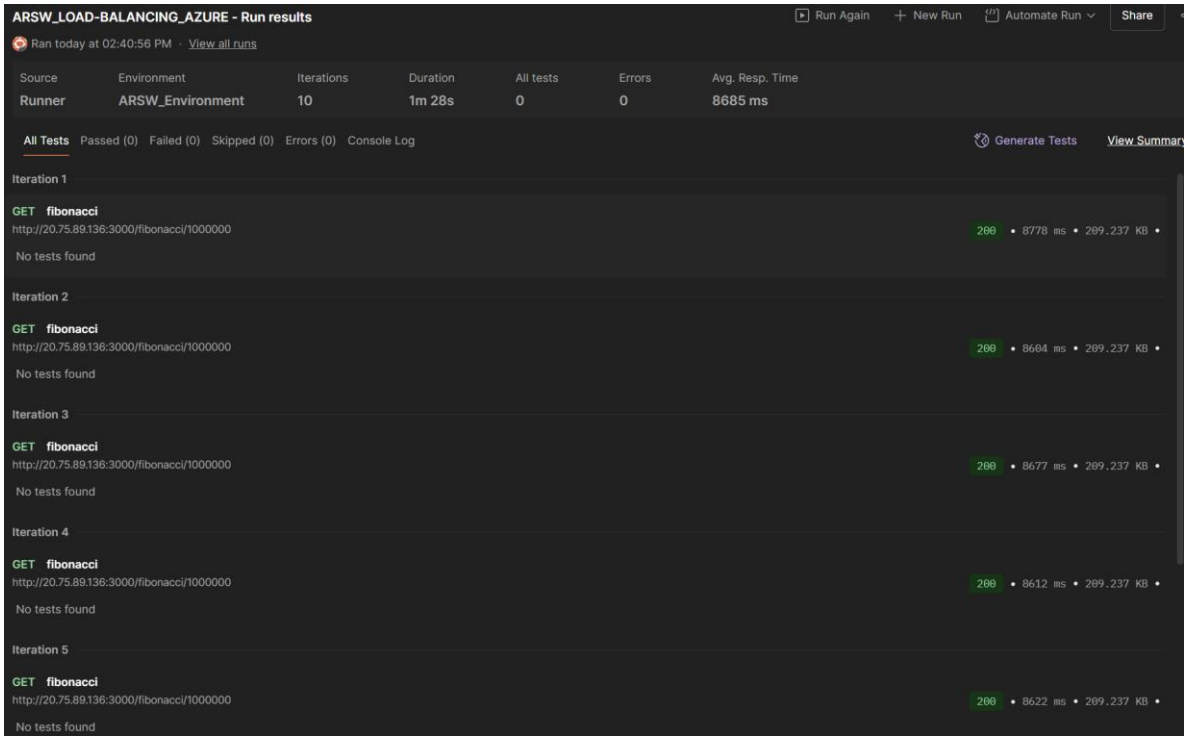
→





11. Una vez el cambio se vea reflejado, repita el paso 7, 8 y 9.

→ Se observa una leve mejora pero no al 100%



12. Evalue el escenario de calidad asociado al requerimiento no funcional de escalabilidad y concluya si usando este modelo de escalabilidad logramos cumplirlo.

→ Tras evaluar el escalamiento vertical a un tamaño B2ms, se observa una mejora marginal en los tiempos de respuesta al duplicar los recursos de cómputo; sin embargo, esta estrategia no logra satisfacer plenamente el requerimiento no funcional de escalabilidad. Aunque el sistema gana capacidad bruta, el cuello de botella persiste debido a la arquitectura monohilo de la aplicación, lo que demuestra que el escalamiento vertical es una solución temporal y costosa.

13. Vuelva a dejar la VM en el tamaño inicial para evitar cobros adicionales.

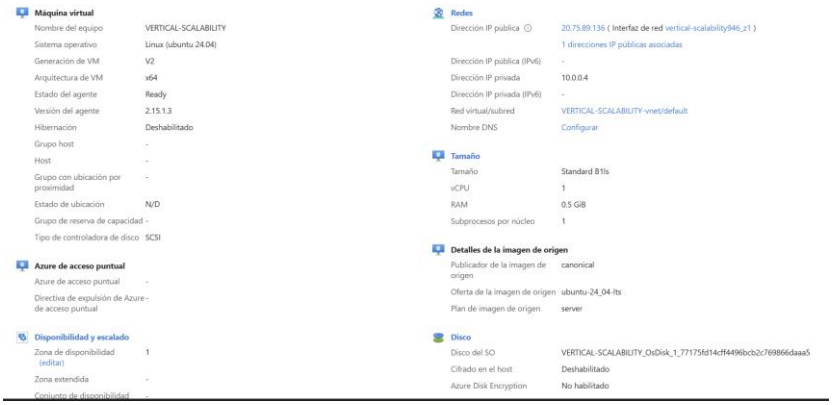
→

Información esencial		Vista JSON
Grupo de recursos (mover)	: SCALABILITY_LAB	
Estado	: En ejecución	
Ubicación	: East US 2 (Zona 1)	
Suscripción (mover)	: Azure for Students	
Id. de suscripción	: 8e09648e-009b-4e99-bbb9-08...	
Zona de disponibilidad	: 1	
Sistema operativo	: Linux (ubuntu 24.04)	
Tamaño	: Standard B1ls (1 vcpu, 0.5 GiB d...	
IP pública de NIC principal	: 20.75.89.136 1 direcciones IP públicas asocia...	
Red virtual/subred	: VERTICAL-SCALABILITY-vnet/d...	
Nombre DNS	: Sin configurar	
Estado de mantenimiento	: -	
Hora de creación	: 21/4/2026, 12:34 a.m. UTC	
Etiquetas (editar)	: Agregar etiquetas	

Preguntas

1. ¿Cuántos y cuáles recursos crea Azure junto con la VM?

→ Son 6 recursos que se crean junto a la VM, son: Interfaz de red, red virtual, subred, grupo de seguridad de red, disco gestionado, dirección ip pública.



Máquina virtual	
Nombre del equipo	VERTICAL-SCALABILITY
Sistema operativo	Linux (ubuntu 24.04)
Generación de VM	V2
Arquitectura de VM	x64
Estado del agente	Ready
Versión del agente	2.15.1.3
Hibernación	Deshabilitado
Grupo host	-
Host	-
Grupo con ubicación por proximidad	-
Estado de ubicación	N/D
Grupo de reserva de capacidad	-
Tipo de controladora de disco	SCSI

Azure de acceso puntual	
Azure de acceso puntual	-
Directiva de expulsión de Azure de acceso puntual	-

Disponibilidad y escalado	
Zona de disponibilidad (editar)	1
Zona extendida	-
Conjunto de disponibilidad	-

Redes	
Dirección IP pública	20.75.89.136 (Interfaz de red vertical-scalability946_z1)
	1 direcciones IP públicas asociadas
Dirección IP pública (IPv6)	-
Dirección IP privada	10.0.0.4
Dirección IP privada (IPv6)	-
Red virtual/subred	VERTICAL-SCALABILITY-vnet/default
Nombre DNS	Configurar

Tamaño	
Tamaño	Standard B1ls
vCPU	1
RAM	0.5 GiB
Subprocesos por núcleo	1

Detalles de la imagen de origen	
Publicador de la imagen de origen	canonical
Oferta de la imagen de origen	ubuntu-24_04-lts
Plan de imagen de origen	server

Disco	
Disco del SO	VERTICAL-SCALABILITY_OsDisk_1_77175fd14cff4496bcb2c769866daaa5
Cifrado en el host	Deshabilitado
Azure Disk Encryption	No habilitado

2. ¿Brevemente describa para qué sirve cada recurso?

→

- **Dirección IP Pública:** Permite el acceso y exposición del servicio a través de internet.
- **Interfaz de Red :** Permite la comunicación de la VM con la red virtual.
- **Red Virtual (VNet):** Define el espacio de direcciones IP privadas donde se aloja la infraestructura.
- **Subred:** Segmentación lógica dentro de la red virtual.
- **Grupo de Seguridad de Red (NSG):** Firewall que controla el tráfico entrante y saliente (ej. apertura de puertos).
- **Disco Gestionado:** Almacenamiento persistente que contiene el sistema operativo y los archivos de la aplicación."

3. ¿Al cerrar la conexión ssh con la VM, por qué se cae la aplicación que ejecutamos con el comando npm FibonacciApp.js? ¿Por qué debemos crear un *Inbound port rule* antes de acceder al servicio?

→ La ejecución directa en la terminal vincula el ciclo de vida del proceso a la sesión SSH. Al cerrar la sesión, el sistema envía una señal de terminación, provocando el apagado de la aplicación. Se requiere el uso de gestores de procesos (como forever) para desvincular el proceso de la sesión de usuario y garantizar su persistencia.

Azure implementa una política de seguridad de 'denegación por defecto'. Para exponer servicios web, es obligatorio configurar reglas de entrada que permitan el tráfico hacia los puertos específicos de la aplicación, garantizando que el firewall no bloquee las peticiones externas.

4. Adjunte tabla de tiempos e interprete por qué la función tarda tando tiempo.

→

Valor de Fibonacci	Tiempo de respuesta (segundos)
1,000,000	13.04 s
1,030,000	47.04 s
1,040,000	59.02 s
1,050,000	72 s
1,060,000	102 s
1,070,000	126 s
1,080,000	108 s
1,090,000	84 s

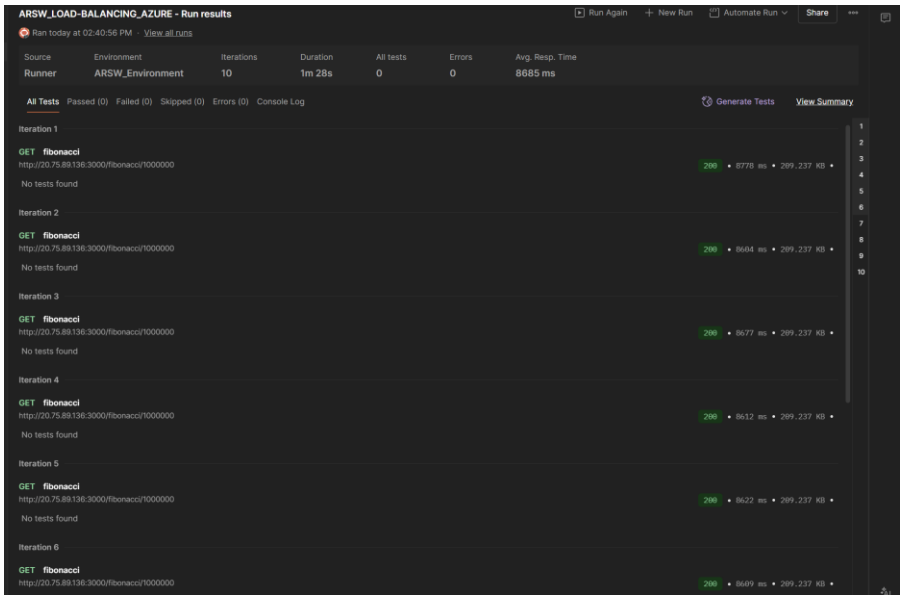
El tiempo de respuesta crece exponencialmente debido a que el cálculo de Fibonacci bloquea el Event Loop de Node.js, impidiendo la concurrencia. La degradación del rendimiento conforme aumenta el valor de entrada confirma que el diseño actual de la aplicación no es escalable.

5. Adjunte imágen del consumo de CPU de la VM e interprete por qué la función consume esa cantidad de CPU.

→ El pico en el uso de CPU registrado en Azure Monitor confirma que la función de Fibonacci actúa como un proceso CPU-bound. La ejecución bloquea el Event Loop de Node.js, saturando el único vCPU disponible en la instancia B1s.

6. Adjunte la imagen del resumen de la ejecución de Postman. Interprete:
    - Tiempos de ejecución de cada petición.
    - Si hubo fallos documentelos y explique.

→ La ejecución en Postman demuestra que el servicio es capaz de procesar las peticiones, pero con una latencia promedio de 8.65 segundos. La ausencia de errores (código 200) indica estabilidad bajo esta carga de 10 iteraciones, sin embargo, el alto tiempo de respuesta confirma que el recurso de cómputo (vCPU) está saturado.



7. ¿Cuál es la diferencia entre los tamaños B2ms y B1ls (no solo busque especificaciones de infraestructura)?

→

Característica	B1ls	B2ms
Capacidad de Ráfaga	Muy limitada; se agota casi instantáneamente con cálculos pesados.	Robusta; permite mantener cargas intensas por más tiempo.
Arquitectura	1 vCPU; el hilo principal queda totalmente bloqueado.	2 vCPU; permite paralelismo de procesos del SO.
Uso ideal	Servidores de desarrollo, tareas cron simples.	Servidores de producción pequeños, APIs con tráfico variable.
Comportamiento bajo estrés	Estrangulamiento (throttling) agresivo.	Rendimiento más predecible y estable.

8. ¿Aumentar el tamaño de la VM es una buena solución en este escenario?, ¿Qué pasa con la FibonacciApp cuando cambiamos el tamaño de la VM?

→ El escalamiento vertical a B2ms mejora marginalmente el tiempo de respuesta al proporcionar más ciclos de CPU, pero no resuelve el cuello de botella arquitectónico. La aplicación sigue siendo monohilo y, ante una carga de usuarios real, el Event Loop se saturará inevitablemente.

9. ¿Qué pasa con la infraestructura cuando cambia el tamaño de la VM? ¿Qué efectos negativos implica?

→El redimensionamiento de la VM implica un proceso de desasignación y migración a un nuevo host físico, lo cual garantiza la actualización de los recursos de cómputo pero introduce un periodo inevitable de inactividad.

10. ¿Hubo mejora en el consumo de CPU o en los tiempos de respuesta? Si/No ¿Por qué?

→ Sí, en términos de saturación. Aunque el cálculo de Fibonacci seguirá consumiendo el 100% del núcleo que utiliza, el impacto de rendimiento es menor porque el sistema ya no queda totalmente bloqueado.

11. Aumente la cantidad de ejecuciones paralelas del comando de postman a 4. ¿El comportamiento del sistema es porcentualmente mejor?

→ En lugar de una mejora porcentual, el tiempo de respuesta total aumenta debido a la contención en el *Event Loop* y al tiempo de espera en la cola de peticiones.